

Presence for students: the second track

A brief for a collaborator taking the same problem from the student's side — studying, goals, routine — on the same engine, the same stack and the same values.

Draft for discussion · 12 September 2026 · read Documents 1 (progress) and 2 (values) first

1. The idea in one paragraph

Presence already has a runtime that runs AI agents on a schedule with a daily spending cap, a tracker engine with conventions, a messenger delivery layer, a configuration model built around confirmed facts, and a charter. All of that is domain-neutral. The careers track points it at job postings; the **student track points the same machine at a degree**: turning a syllabus and a timetable into a plan, keeping a daily study routine, tracking goals and deadlines, and coaching — delivered as a bot in the student's messenger, running on the student's own laptop, free. One engine, two tracks, one brand.

2. What is shared and what is separate

Layer	Careers track (built)	Student track (to build)
Runtime shared	Agent executor (Anthropic / OpenAI / Ollama), scheduler, daily token budget, config loader, secrets — reused as-is. The student track never re-implements these.	
Messenger shared	Telegram adapter (W4, next) with pairing, daily brief, commands. One bot can serve both tracks or each track its own — decide together.	
Tracker engine shared	Jobs: company, title, url, status, events timeline, <code>extra</code> .	Records: modules, assignments, exams, goals — same design (small fixed core, events timeline, <code>extra</code>), a second schema in the same engine. Status vocabulary is the track's own (e.g. <code>planned · in_progress · submitted · graded · missed</code>).
Agents per track	Scout (find & judge postings), Brief (daily digest).	Planner (syllabus + timetable + goals → a weekly plan), Brief (today's study brief: what, when, one first action), Coach (spaced-repetition questions, quizzes, retrospectives) — the pipeline's existing Coach and training hub are the precedent.
Connectors per track	LinkedIn search (library-based).	Deterministic imports first: ICS calendars (university timetables export them), Moodle/Canvas exports, reading lists, a pasted syllabus. No scraping needed — a strong first connector because it is entirely under the student's control.

Onboarding per track	CV upload + five questions.	Timetable/syllabus upload + five questions: goals for the term, hours available per week, exam dates, weak subjects, routine constraints (sleep, work shifts). Settings remain a conversation, never a form.
Packs shared format	Prompts, drills and sims are plain-text packs (MIT). A community pack library serves both tracks; a "CCAT drill" pack and a "linear algebra recall" pack are the same kind of thing.	
Charter & values shared	Identical. For students two rules bite hardest: no invented facts (never fabricate a grade, a deadline or a "you're on track" that isn't true) and data stays home (academic records are sensitive; nothing leaves the laptop).	

3. Proposed repository model

One core, two domain packages, one repository. The runtime, messenger and tracker engine stay in `src/presence/`; each track owns a domain package (`src/presence/domains/careers/`, `src/presence/domains/study/`) holding its agents, schema, connectors and onboarding. Ownership is explicit (CODEOWNERS): the collaborator owns `domains/study/` outright and proposes changes to the core through pull requests like anyone else.

Why not a fork or a second repo: the tracks would drift apart within weeks, and every runtime fix would need doing twice. Why not one blended product: the student and the job seeker want different daily loops; separate domain packages keep each loop simple. A later option, once the core is stable and packaged, is to publish it and let the study track depend on it as a library — the domain-package boundary makes that a move, not a rewrite.

4. Working agreement

- **Values first.** Document 2 applies in full; the charter is enforced by tests in the shared core.
- **Same toolchain.** Python 3.12, uv, pydantic, SQLite, ruff, pytest, GitHub Actions; conventional commits; minimal, product-focused commit messages; one logical change per pull request; tests with behaviour changes; no network calls in tests.
- **Same milestone shape.** M1 (a config-driven agent runs) is already done for both tracks — reuse it. The student track's own milestones: **M2 — Planner fills a study tracker** from an ICS import; **M3 — it talks to you** (daily study brief in Telegram, first dogfood week by the collaborator); **M4 — anyone can install it**; **M5 — v1.0**.
- **Dogfooding is mandatory.** The collaborator runs the student track on their own term before anyone else does — user number one, exactly as with the careers track.
- **Cadence.** One short weekly sync; decisions written down in the repo (docs/decisions) so neither track depends on memory or chat history.

5. First four weeks for the collaborator

1. **Week 1 — onboard.** Clone, run the environment doctor, run the test suite, run the live check (`python -m presence.core.check ollama <model>` needs no key). Read the architecture spec and Documents 1–2. Deliverable: a one-page **student-track spec** in the same style as the careers spec — the daily loop and the five onboarding questions.

2. **Week 2 — the study tracker.** Define the record core and status vocabulary; implement it on the shared engine; validate with the collaborator's own real timetable, read-only, exactly as the careers tracker was validated on a real 612-row sheet.
3. **Week 3 — first connector.** ICS calendar import behind the shared connector interface, with tests on recorded fixtures. Deliverable: M2 (Planner fills a study tracker).
4. **Week 4 — Planner + Brief packs.** The two prompts in the shared pack format; a first eval set (fixture syllabi with expected plans) so quality is measured across models, not guessed.

6. Questions to decide together

- One bot for both tracks, or one per track? (Affects pairing and command namespaces.)
- Name and brand: "Presence" with tracks, or a named sub-brand for students? The brand-book being started now should answer this once.
- Which university systems the collaborator can export from (ICS is universal; Moodle/Canvas vary) — this decides the first connector's scope.
- How the Coach uses the shared training studio when it arrives (drills for exams and drills for assessments are one mechanism).
- Contribution credit and licensing: contributors license under the file's governing licence (AGPL core, MIT packs) — confirm the collaborator is comfortable with that before the first commit.

The one-sentence pitch for the collaborator: the hard, generic part — an agent runtime that is safe, capped, local and honest — already exists and is tested; your job is the part only a student can get right: what a good study day looks like, and how a companion should nudge without nagging.